

Simulation-Grouped Per-Class QoS Prediction for SD-WAN

Radhika (Rishiv) Shitlani
 School of Computer Science
 University of Galway, Galway, Ireland

Abstract

Software-defined wide area networks (SD-WANs) must treat traffic classes differently, yet a single strong overall accuracy score can mask poor performance for the class that most needs scrutiny: best-effort traffic. This paper addresses two related tasks: predicting per-flow delay and detecting service-level-agreement (SLA) violations. We use 29,280 flow records from 400 BNNetsimulator runs, covering Gold, Silver, and Bronze traffic under three scheduling policies. An XGBoost regressor is evaluated with five-fold cross-validation grouped by simulation identifier, ensuring that no simulation run contributes flows to both sides of a fold. Under this grouped protocol, the model achieves an overall log-delay R^2 of 0.747, with class-specific values of 0.907, 0.898, and 0.680 for Gold, Silver, and Bronze; the corresponding SLA-violation F1 scores are 0.940, 0.680, and 0.554. A conventional row-wise split reports a higher R^2 of 0.766, consistent with modest optimism from shared simulation context across folds. The same grouped protocol shows that 90th-percentile delay is only moderately predictable ($R^2 = 0.293$ overall) and that jitter is not predicted reliably ($R^2 = -0.040$). These results indicate that grouped, per-class evaluation is important for credible QoS prediction when multiple dependent flows originate from each simulation run.

Index Terms

SD-WAN, quality of service, machine learning, XGBoost, grouped cross-validation, SLA prediction

I. INTRODUCTION

Software-defined wide area networking (SD-WAN) separates network control from packet forwarding, giving operators centralized control over routing and QoS policy across heterogeneous links. In practice, however, this control is typically exercised too late: operators adjust queue weights or reroute traffic only after congestion has already affected users. Predicting QoS ahead of an SLA breach could make this process proactive rather than reactive, but only if the prediction can be trusted for the traffic that most needs protecting. Gold, Silver, and Bronze traffic differ in priority, in how their delay behaves, and in what an error actually costs, so a model that appears accurate on average can still be unreliable for exactly the class most exposed to congestion.

Machine learning has already found several applications in SD-WAN research, including predicting how many virtual network functions a network will require [1], routing traffic based on application type [2], and steering traffic engineering with reinforcement learning [3], [4]. Other work optimises SD-WAN

routes and queueing policies directly from measured QoS [5], [6]. These studies demonstrate that predictive or adaptive control is feasible, but none directly answer a narrower, more practical question: how accurately can continuous delay and SLA violations be predicted, separately for each traffic class, from a compact and reproducible feature set?

A second, less obvious problem is evaluation leakage. Each packet-level simulation run produces many flow rows at once, so randomly splitting individual rows can place flows from the same run on both sides of a train/validation split. The simulation identifier is never used as an input feature, but the shared topology, load, and scheduler conditions behind those rows can still make the validation task artificially easy. This paper therefore treats the simulation run, rather than the individual flow row, as the unit of evaluation.

This paper makes three contributions:

- a reproducible per-class delay-prediction study using 29,280 flows from 400 BNNetSimulator runs;
- a simulation-grouped evaluation reporting both regression and SLA-trigger performance separately for Gold, Silver, and Bronze traffic, with row-wise cross-validation retained only as a sensitivity comparison; and
- an extension of the grouped protocol to two secondary QoS targets, tail (90th-percentile) delay and jitter, establishing the limits of what a static configuration feature set can predict.

II. RELATED WORK

Machine learning has been applied to several adjacent network-management tasks, though rarely to per-class QoS prediction itself. Rahman *et al.* predict how many virtual network functions a system will require as load changes [1], while Zheng *et al.* combine traffic classification with QoS-aware routing decisions [2]; both use machine learning to inform a decision rather than to predict a continuous QoS outcome per class. For SD-WAN specifically, Quang *et al.* formulate joint routing and QoS policy optimisation [5] and later add passive available-bandwidth estimation to a two-loop controller [6]; these are optimisation systems built around a fixed queueing model rather than learned predictors evaluated against ground truth. Recent work also explores safe deep reinforcement learning for load balancing [3] and multi-objective, graph-based traffic engineering [4]. Among these reviewed studies, none reports how well delay or SLA compliance can be predicted separately for each priority class, which is the specific gap this paper addresses.

When labelled enterprise telemetry is unavailable, network simulators offer a reproducible alternative. BNNetSimulator generates packet-level data compat-

ible with the modelling methodology behind RouteNet-Fermi [7], [8], and this study uses it to examine a narrower, more practical question than the wider literature addresses: can static configuration and path features alone support reliable, class-specific QoS prediction? XGBoost is used as the primary model because gradient-boosted trees remain strong performers on small-to-medium tabular datasets [9], [10], train comparatively quickly, provide inexpensive split-based feature-importance measures, and support efficient Tree SHAP explanations.

III. METHODOLOGY

A. Dataset and QoS Classes

The dataset was generated with the official BNNetSimulator container [7] and contains 29,280 source-destination flow records from 400 simulation runs, with 30–132 flows per run. Traffic is divided into three classes: Gold (4,592 rows, highest priority), Silver (8,991 rows, interactive traffic), and Bronze (15,697 rows, best effort). The experiments span strict priority, WFQ, and deficit round robin scheduling across four scenarios: fixed WFQ weights, randomised WFQ profiles, and two mixed-policy scenarios with skewed and equal class proportions. The generator was configured to avoid deliberate near-saturation, so the results below should be read as a controlled study of per-class prediction rather than a stress test of a saturated production network.

Each input row includes the traffic class, offered bandwidth, traffic distribution, route length, network size, maximum average load, link bandwidth, scheduling policy, class queue weight, and minimum class weight. The prediction target is the natural logarithm of average delay in seconds. Any field that would leak the answer, namely raw delay, jitter, packet loss, delay percentiles, achieved bandwidth, the scenario label, and the simulation identifier itself, is excluded from the feature matrix. Missing numeric values are median-imputed and categorical variables are one-hot encoded, both computed separately within each training fold to avoid leakage.

B. Prediction and Grouped Evaluation

The primary regressor is XGBoost, configured with 200 trees, a maximum depth of four, a learning rate of 0.05, and row and feature subsampling both set to 0.9. Because the model is trained on log delay, predictions are converted back to milliseconds before being reported, so that the error values carry operational meaning. Performance is measured using mean absolute error (MAE) on the millisecond scale and R^2 on log delay.

Five-fold cross-validation uses scikit-learn’s GroupKFold with *simulation_id* as the grouping key, keeping every run’s flows on one side of each fold. As a sensitivity check, the same model is also evaluated with a conventional shuffled, row-wise five-fold split, allowing the two protocols to be compared directly. Preprocessing and model fitting are refit independently within every fold.

Continuous delay predictions are also converted into binary SLA-violation triggers, using thresholds of 30, 50, and 60 ms for Gold, Silver, and Bronze respectively. Precision indicates how much a trigger can be trusted once it fires, recall indicates how many real violations are actually caught, and F1 balances the two. The Bronze threshold was derived from an exploratory sensitivity sweep rather than a fixed requirement, and should be read as a reasonable operating point rather than a universal SLA value.

The same grouped folds and leakage-safe feature exclusions are reused for two secondary targets, 90th-percentile delay and jitter, evaluated directly in milliseconds since that is the scale most relevant to the operational question. A negative R^2 in this context does not indicate an error; it indicates that the model’s predictions generalise worse than simply predicting the validation-fold mean.

C. Reproducibility

Data generation, preprocessing, grouped evaluation, and plotting are implemented as separate, reusable Python scripts. Randomised estimators use a fixed seed (42), and every reported metric is derived from out-of-fold predictions rather than training predictions, ensuring that no result is inflated by the model having already seen the row it is scored on. The grouped evaluator also records which fold unit (row or simulation run) produced each result, to prevent the two from being conflated during reporting. The generated dataset, experiment scripts, and result tables are available in the project repository; a permanent archival identifier will be added for the camera-ready version.

IV. RESULTS

A. Per-Class Delay and SLA Prediction

Table I reports the primary grouped results. Overall R^2 is 0.747, with a delay-scale MAE of 13.19 ms, though this single figure conceals a substantial class-dependent gap. Gold and Silver are predicted accurately ($R^2 = 0.907$ and 0.898), while Bronze falls to 0.680 and its MAE nearly triples to 21.57 ms. This pattern is consistent with protected traffic being shaped primarily by stable topology and scheduler configuration, which the static features capture

TABLE I
FIVE-FOLD SIMULATION-GROUPED RESULTS

Class	MAE (ms)	R^2	Prec.	Recall	F1
Gold	3.17	0.907	0.986	0.898	0.940
Silver	3.67	0.898	0.798	0.593	0.680
Bronze	21.57	0.680	0.569	0.540	0.554

TABLE II
SENSITIVITY TO THE CROSS-VALIDATION UNIT

Protocol	Overall R^2	Gold R^2	Silver R^2	Bronze R^2
Row K-fold	0.766	0.914	0.908	0.704
Grouped by run	0.747	0.907	0.898	0.680

well, while best-effort delay may also depend on transient contention that these features cannot observe.

Gold’s SLA trigger maintains a very high precision of 0.986: 98.6% of predicted Gold violations are true violations (1,184 of 1,201). This is an important property for any controller that acts on such predictions, since false positives on the highest-priority class could trigger unnecessary intervention. Silver and Bronze are noticeably less reliable, particularly on recall, and Bronze’s F1 of 0.554 illustrates an important point: a regressor that appears accurate in aggregate does not automatically translate into a strong SLA-violation detector for best-effort traffic.

B. Effect of Grouping by Simulation

Table II and Fig. 1 compare the grouped protocol against a conventional shuffled row-wise split. Grouping reduces overall R^2 from 0.766 to 0.747, with the largest drop occurring in Bronze, from 0.704 to 0.680, and its SLA F1 falling from 0.592 to 0.554; Gold is largely unaffected. This reduction is consistent with modest optimism from sharing simulation context across row-wise folds and supports treating the simulation run, rather than the row, as the evaluation unit for this dataset.

An exploratory row-wise comparison found nearly identical overall scores for XGBoost ($R^2 = 0.766$), a multilayer perceptron (0.766), and an FT-Transformer (0.762). These neural results have not yet been repeated under simulation-grouped folds, so they are reported here only as supporting evidence rather than the paper’s primary generalisation estimate, but they are sufficient to justify XGBoost as the lower-cost model of choice for the remainder of the study.

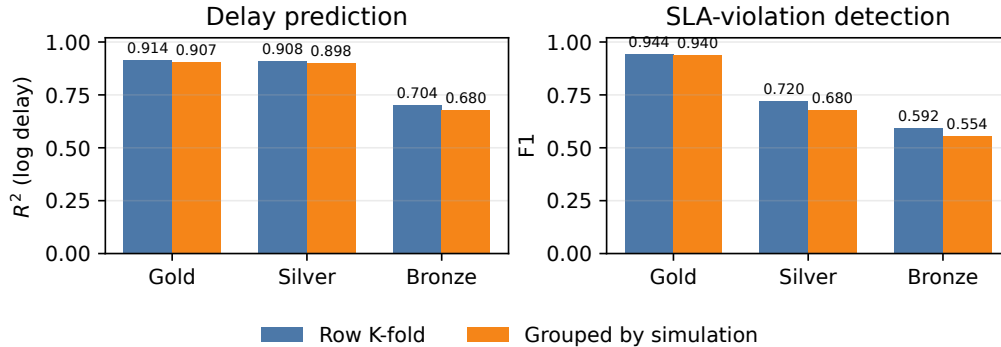


Fig. 1. Row-wise and simulation-grouped performance by QoS class.

TABLE III
EXPLORATORY ROW-WISE MODEL COMPARISON

Model	Overall R^2	Gold R^2	Silver R^2	Bronze R^2
XGBoost	0.766	0.914	0.908	0.704
MLP	0.766	0.912	0.908	0.705
FT-Transformer	0.762	0.900	0.898	0.703

C. Model Complexity and Feature Evidence

Table III breaks this exploratory comparison down by class. All three model families reach essentially the same Bronze ceiling under the row-wise protocol, and the FT-Transformer trails only marginally on Gold and Silver, with no gap exceeding 0.014 R^2 . This is not proof of equivalence under grouped validation, but it is a clear signal that additional neural complexity provides little benefit here, since it cannot compensate for short-timescale traffic information absent from the feature set.

SHAP values for the final XGBoost model support this conclusion (Fig. 2). Route length dominates: additional hops push predictions toward higher log delay, while shorter paths pull them down. Network size and traffic class follow at some distance, with queue weights and scheduling-policy indicators contributing comparatively little on their own. These represent associations learned within the simulator rather than causal estimates, but they tell a consistent story: the model relies primarily on stable path structure. This structure is genuinely informative for protected Gold and Silver flows, but the weaker Bronze results suggest that it does not fully capture transient contention affecting best-effort delay.

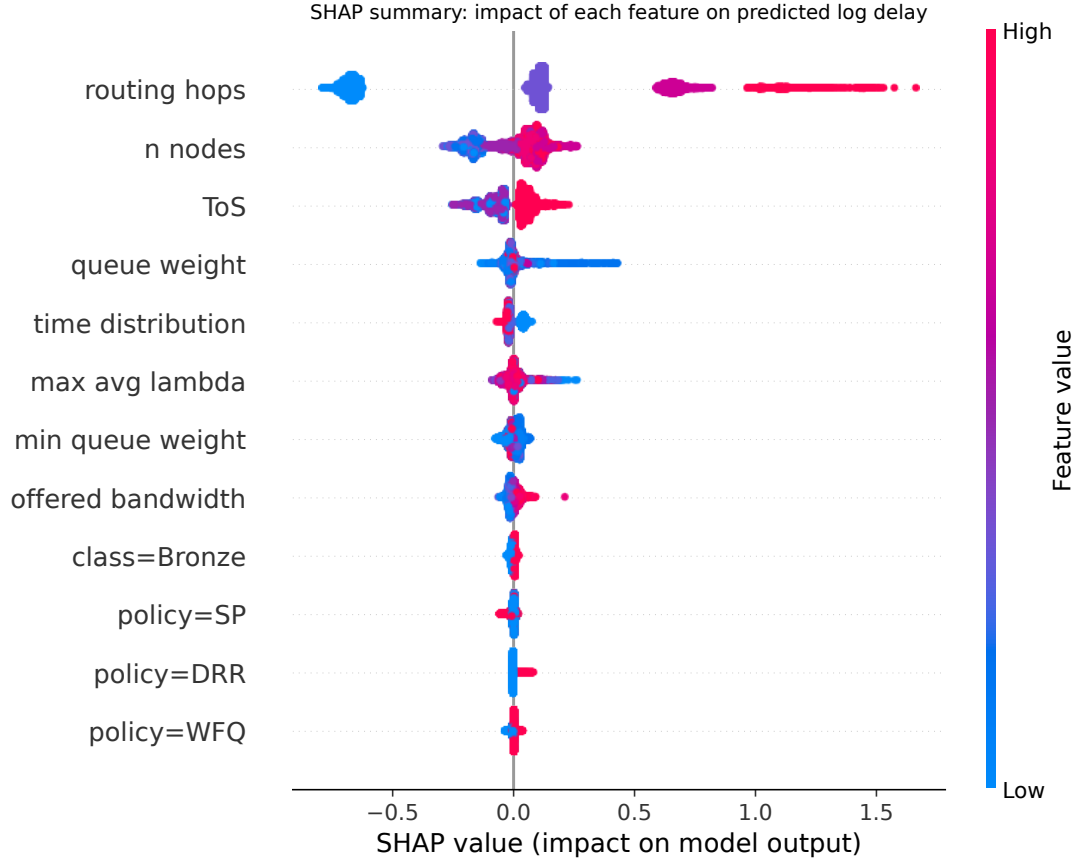


Fig. 2. SHAP summary for the XGBoost log-delay model; colour denotes feature value.

D. Tail Delay and Jitter

Table IV applies the same grouped protocol to two secondary QoS targets. Tail delay is noticeably harder to predict than average delay: overall R^2 drops to 0.293, reaching only 0.550 and 0.490 for Gold and Silver, and falling further to 0.269 for Bronze, whose MAE climbs to 51.81 ms. This result is consistent with rare contention events influencing the upper tail, although those events are not directly measured by the current features.

Jitter does not generalise from the static feature set at all: overall R^2 is -0.040 , and every class is negative. Gold and Silver exhibit small absolute MAE, but their within-class variance is so low that even modest errors exceed the error of simply predicting the validation-fold mean. Together, these results clarify the boundary of what this feature set can convey: it describes average delay well, tail delay only partially, and moment-to-moment delay variation not at all.

TABLE IV
GROUPED RESULTS FOR SECONDARY QoS TARGETS

Target	Class	MAE (ms)	R^2
Delay p90	Overall	32.51	0.293
	Gold	8.92	0.550
	Silver	10.87	0.490
	Bronze	51.81	0.269
Jitter	Overall	3.22	-0.040
	Gold	0.51	-7.555
	Silver	1.01	-13.182
	Bronze	5.28	-0.034

V. DISCUSSION AND LIMITATIONS

The central finding here is not simply that delay is predictable, but that its predictability depends on the traffic class, the target, and the validation design employed. Route length, network size, and traffic class encode stable structure, and this structure explains protected average delay well. Bronze, having the lowest service priority, may be more exposed to bursts and queue occupancy that the current features do not measure. The poor jitter results reinforce this interpretation: increasing model complexity cannot recover transient state that is absent from the features.

Several limitations should temper these claims. First, the data is simulator-generated and was configured without deliberate near-saturation; grouped folds test generalisation to unseen runs from the same generator, not transfer to a real enterprise SD-WAN. Second, the feature set is entirely static: telemetry such as instantaneous queue occupancy, recent arrival-rate windows, and recent loss would be required to model transient behaviour, and none of this is present here. Third, the SLA thresholds used are experimental operating points rather than contractual values drawn from a real deployment. Future work should repeat every model comparison under grouped outer folds, evaluate under heavier congestion, and validate against real controller telemetry.

VI. CONCLUSION

This paper presented a simulation-grouped, per-class evaluation of QoS prediction for SD-WAN-like networks. XGBoost generalises well to unseen simulation runs for Gold and Silver average delay, is noticeably less reliable for Bronze, and does not predict jitter reliably. In this dataset, row-wise cross-validation produces modestly higher scores than simulation-grouped validation, supporting the use of the simulation run rather than the individual flow as the

evaluation unit. The broader implication is straightforward: trustworthy AI-assisted QoS prediction requires class-aware metrics, group-safe validation, and an honest account of what static network features can and cannot support.

REFERENCES

- [1] S. Rahman, T. Ahmed, M. Huynh, M. Tornatore, and B. Mukherjee, “Auto-scaling vnfs using machine learning to improve qos and reduce cost,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2018, pp. 1–6.
- [2] W. Zheng, M. Yang, C. Zhang, Y. Zheng, Y. Wu, Y. Zhang, and J. Li, “Application-aware qos routing in sdns using machine learning techniques,” *Peer-to-Peer Networking and Applications*, vol. 15, no. 1, pp. 529–548, 2022.
- [3] L. Dinh, P. T. A. Quang, and J. Leguay, “Towards safe load balancing based on control barrier functions and deep reinforcement learning,” *arXiv preprint arXiv:2401.05525*, 2024.
- [4] Y. Chen, A. Gu, L. Cui, and L. Lin, “MTEAL: Network routing optimization of SD-WAN traffic engineering integrating multi-dimensional QoS metrics,” *Journal of Network and Computer Applications*, vol. 242, p. 104272, 2025.
- [5] P. T. A. Quang, X. Gong, C. Liu, K. Li, J. Leguay, X. Zhang, Y. Zhang, J. Li, and K. Ye, “Routing and qos policy optimization in sd-wan,” *arXiv preprint arXiv:2209.12515*, 2022.
- [6] P. T. A. Quang, J. Leguay, X. Gong, and H. Xu, “Global qos policy optimization in sd-wan,” in *Proc. IEEE 9th Int. Conf. Network Softwarization (NetSoft)*, 2023, pp. 202–206.
- [7] BNN-UPC, “BNNetSimulator: A packet-level network simulator to generate datasets for research and analysis,” GitHub repository: <https://github.com/BNN-UPC/BNNetSimulator>, 2023.
- [8] M. Ferriol-Galmés, J. Paillisse, J. Suárez-Varela, K. Rusek, S. Xiao, X. Shi, X. Cheng, P. Barlet-Ros, and A. Cabellos-Aparicio, “Routenet-fermi: Network modeling with graph neural networks,” *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 3080–3095, 2023.
- [9] L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?” in *Advances in Neural Information Processing Systems*, vol. 35, 2022.
- [10] R. Shwartz-Ziv and A. Armon, “Tabular data: Deep learning is not all you need,” *Information Fusion*, vol. 81, pp. 84–90, 2022.